

Daily Quiz Challenge

Autoři:

Hynek Faktor, Jakub Šrámek, Dan Oujeský,

David Tesař, Nguyen Quoc Khanh

2026

Úvod

Tento projekt se zaměřuje na vytvoření webové aplikace ve formě každodenního kvízu pro studenty. Aplikace bude zábavnou a soutěživou platformou, která motivuje uživatele k pravidelné aktivitě a umožní jim porovnávat své znalosti a rychlost reakcí s ostatními.

Projekt je navržen tak, aby byl vhodný pro školní prostředí.

1. Základní informace o projektu

Členové týmu:

David Tesař - Scrum Master

Nguyen Quoc Khanh - Sysadmin

Dan Oujeský - Backend Developer

Jakub Šrámek - Frontend Developer

Hynek Faktor - Dokumentarista

Cílová skupina

Aplikace je určena především pro:

- studenty SPŠE Ječná
- případně učitele (např. pro administraci nebo testování)

Základní princip fungování

Každý registrovaný a přihlášený uživatel bude mít možnost se **jednou denně** zúčastnit kvízu.

Průběh kvízu:

1. Uživatel si zvolí **jednu ze tří obtížností**:
 - a. Lehká
 - b. Střední
 - c. Těžká
2. Každý kvíz bude obsahovat **3 otázky**.
3. Za správné odpovědi získá uživatel body. Počet bodů bude záviset na:
 - a. zvolené obtížnosti (vyšší obtížnost = více bodů),

- b. rychlosti odpovědi (rychlejší odpověď = více bodů).
- 4. Po dokončení kvízu se výsledek uloží do databáze a promítne se do žebříčků.

Bodovací systém

Body budou přidělovány na základě dvou hlavních faktorů:

- **Obtížnost kvízu**
 - Lehká – nejnižší bodový zisk
 - Střední – střední bodový zisk
 - Těžká – nejvyšší bodový zisk
- **Rychlost odpovědi**

Čím rychleji uživatel odpoví správně, tím více bodů získá.

žebříček (Leaderboard)

- Zobrazuje hráče s nejvyšším celkovým počtem bodů.
- Body se sčítají dlouhodobě.

Žebříčky podporují soutěživost mezi studenty a motivují je k pravidelnému hraní.

Hlavní funkce aplikace

- Registrace a přihlášení uživatelů
- Omezení na 1 odehraný kvíz denně
- Výběr obtížnosti kvízu
- Zobrazení otázek a možnost odpovědi
- Měření času odpovědi
- Výpočet bodů podle obtížnosti a rychlosti
- Ukládání výsledků do databáze
- Zobrazení celkového žebříčku
- Administrace otázek (správa otázek oprávněným uživatelem)

2. Cíl aplikace

Cílem aplikace je:

- vytvořit jednoduchý, ale zábavný kvízový systém,
- podpořit každodenní zapojení studentů,
- zavést soutěžní prvek pomocí bodování a žebříčků,
- nabídnout přehledné a rychlé herní relace (krátké kvízy).

3. Stakeholders

Student (uživatel) - Hlavní uživatel aplikace; Chce se bavit, soutěžit a sbírat body

Vývojový tým - Studenti vyvíjející aplikaci; Úspěšné dokončení projektu

4. Use-cases

UC1: Registrace uživatele - Uživatel dostane vygenerovaná 3 slova, a je automaticky uložen

UC2: Přihlášení uživatele - Uživatel se přihlásí do systému (použije 3 slova vygenerovaná při registraci)

UC3: Změna jména uživatele - Systém uživateli umožňuje změnu jména

UC4: Vyplnění kvízu - Uživatel odpoví na 3 otázky

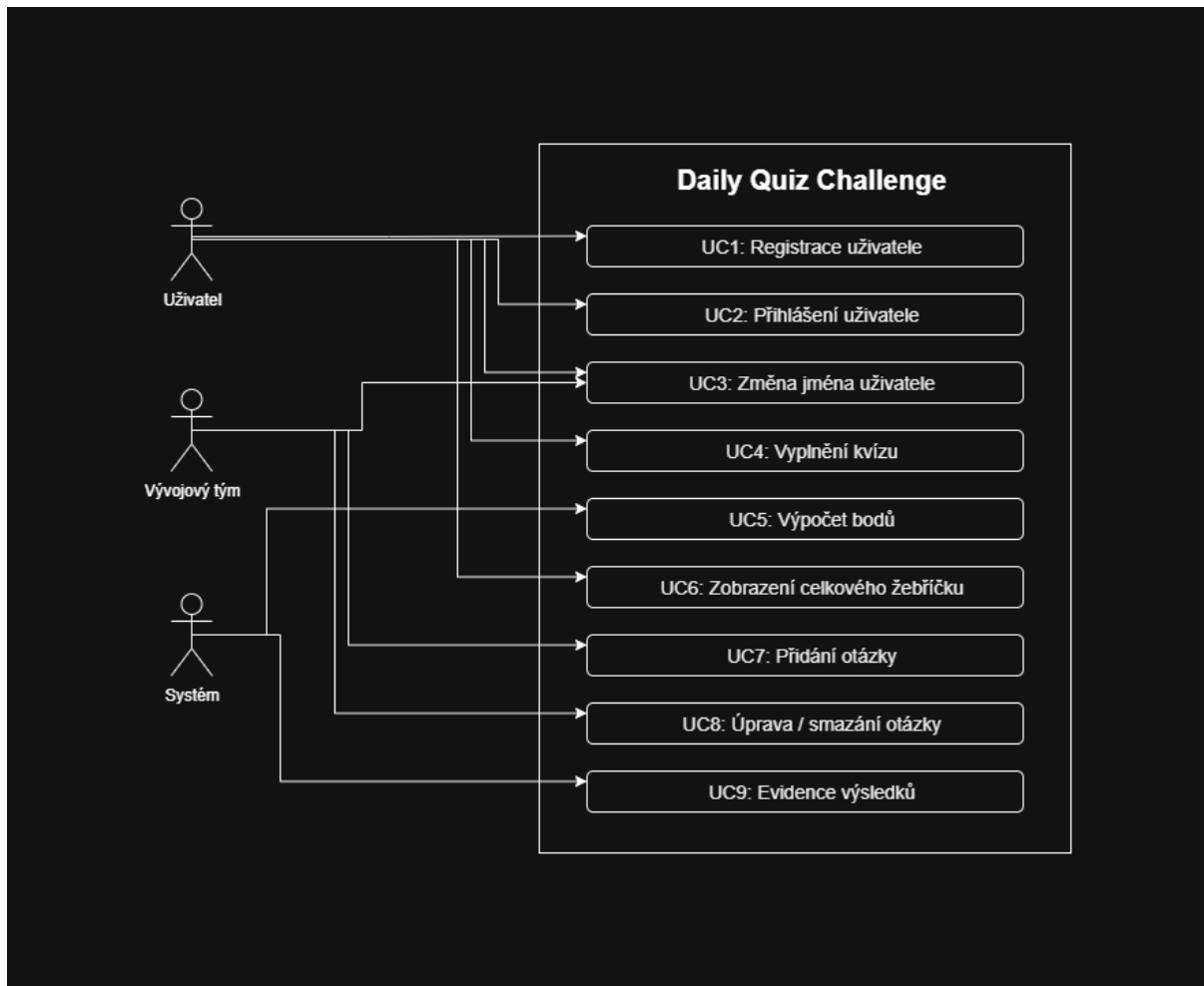
UC5: Výpočet bodů - Systém spočítá body podle obtížnosti a času

UC6: Zobrazení celkového žebříčku - Uživatel vidí celkové pořadí hráčů

UC7: Přidání otázky - Správce vloží novou otázku

UC8: Úprava / smazání otázky - Správce upraví nebo odstraní otázku

UC9: Evidence výsledků - Systém ukládá výsledky do databáze



5. Požadavky

REQ-FUNC-1: Registrace uživatele

- Popis: Systém uživatelům vygeneruje 3 slova, a tím se vytvoří účet
- Typ: Funkční
- Zdroj: UC1 - Registrace uživatele
- Ověření: Úspěšná registrace několika účtů

REQ-FUNC-2: Přihlášení uživatele

- Popis: Systém umožní registrovaným uživatelům přihlášení
- Typ: Funkční
- Zdroj: UC2 - Přihlášení uživatele
- Ověření: Úspěšné přihlášení uživatele k existujícímu účtu

REQ-FUNC-3: Denní limit kvízu

- Popis: Uživatel může odehrát pouze jeden kvíz denně

- Typ: Funkční
- Zdroj: UC4 - Vyplnění kvízu
- Ověření: Po vyplnění jednoho kvízu nelze vyplnit další ze stejného účtu

REQ-FUNC-4: Volba obtížnosti

- Popis: Uživatel si může vybrat obtížnost (lehká, střední, těžká)
- Typ: Funkční
- Zdroj: UC4 - Vyplnění kvízu
- Ověření: Je možné vybrat si mezi obtížnostmi

REQ-FUNC-5: Zobrazení otázek

- Popis: Systém zobrazí uživateli 3 otázky v rámci kvízu
- Typ: Funkční
- Zdroj: UC4 - Vyplnění kvízu
- Ověření: Úspěšné zobrazení otázek

REQ-FUNC-6: Záznam odpovědi

- Popis: Systém zaznamená odpovědi uživatele
- Typ: Funkční
- Zdroj: UC9 - Evidence výsledků
- Ověření: Zaškrtnuté odpovědi uživatelem se zaznamenávají

REQ-FUNC-7: Měření času

- Popis: Systém měří čas odpovědi na každou otázku
- Typ: Funkční
- Zdroj: UC5 - Výpočet bodů
- Ověření: Čím rychleji uživatel odpoví, tím více bodů za odpověď dostane

REQ-FUNC-8: Výpočet bodů

- Popis: Systém vypočítá body podle obtížnosti a rychlosti
- Typ: Funkční
- Zdroj: UC5 - Výpočet bodů
- Ověření: Aplikace po vyplnění kvízu úspěšně přidá uživateli body

REQ-FUNC-9: Uložení výsledku

- Popis: Systém uloží výsledek kvízu do databáze

- Typ: Funkční
- Zdroj: UC9 - Evidence výsledků
- Ověření: Nahrané body uživatelem se úspěšně ukládají

REQ-FUNC-10: Změna jména uživatele

- Popis: Systém uživateli umožňuje změnu jména
- Typ: Funkční
- Zdroj: UC3 - Změna jména uživatele
- Ověření: Úspěšné změnění svého jména uživatelem

REQ-FUNC-11: Celkový žebříček

- Popis: Systém zobrazí celkový žebříček hráčů
- Typ: Funkční
- Zdroj: UC6 - Zobrazení celkového žebříčku
- Ověření: Možné zobrazení žebříčku

REQ-QUAL-1: Dostupnost

- Popis: Aplikace bude dostupná online 24/7 (kromě údržby)
- Typ: Kvalitativní
- Ověření: Aplikace je stále dostupná

REQ-QUAL-2: Přehlednost UI

- Popis: Uživatelské rozhraní bude jednoduché a srozumitelné
- Typ: Kvalitativní
- Ověření: UI je přehledné

REQ-QUAL-3: Rychlá odezva

- Popis: Odezva aplikace na akce uživatele bude do 2 sekund
- Typ: Kvalitativní
- Ověření: Aplikace reaguje do 2 sekund

REQ-QUAL-4: Kompatibilita

- Popis: Aplikace bude funkční v běžných moderních prohlížečích

- Typ: Kvalitativní
- Ověření: Možné zobrazení žebříčku

REQ-QUAL-5: Školní vhodnost

- Popis: Obsah aplikace nebude obsahovat nevhodný nebo urážlivý materiál
- Typ: Kvalitativní
- Ověření: V aplikaci se skutečně nic závadného nenachází

6. Rozhodnutí o technologiích

Při návrhu aplikace jsme volili technologie s ohledem na jejich rozšířenost, uplatnění v praxi a vhodnost pro tvorbu webových aplikací.

Backend

Technologie	Rozšířenost	Uplatnění	Bezpečnost	Výkon	Náročnost	Celkem
Symfony	3*5	4*5	5*5	4*4	2*4	86
Laravel	3*5	4*5	5*4	4*3	2*4	75
Django	3*4	4*4	5*5	4*3	2*3	71

Server

Technologie	Rozšířenost	Výkon	Konfigurace	Funkce	Náročnost	Celkem
Nginx	5*5	5*5	4*4	4*4	3*4	93
Apache	5*5	3*4	4*5	5*5	3*3	85
Caddy	3*4	4*5	5*5	4*4	4*4	86

Backend – Symfony (PHP framework)

Pro serverovou část aplikace jsme zvolili framework **Symfony**. Důvody této volby jsou následující:

- **Rozšířenost v praxi** – Symfony patří mezi nejpoužívanější PHP frameworky, zejména v profesionálním a korporátním prostředí.

- **Uplatnění v budoucnu** – Znalost Symfony je dobře využitelná na trhu práce, protože mnoho firem na něm staví své interní i veřejné systémy.
- **Robustnost a škálovatelnost** – Framework je vhodný pro větší a strukturované projekty, podporuje čistou architekturu a oddělení logiky aplikace.
- **Bezpečnost** – Symfony obsahuje vestavěné nástroje pro práci s uživateli, autentizaci, autorizaci a ochranu proti běžným webovým útokům.
- **Náročnost** – Symfony byla ideální volba, kvůli předešlým úkolům na WA
- **Výkon** – Symfony je výkonné hlavně v produkci

Volbou Symfony si tým zároveň prohlubuje znalosti technologie, která je relevantní pro reálnou praxi, nejen pro školní projekty.

Infrastruktura a nasazení

Pro nasazení aplikace byla zvolena VPS (Virtual Private Server) na platformě Oracle Cloud.

Důvody této volby:

- Plná kontrola nad serverem – možnost konfigurace Nginx, PHP a dalších služeb.
- Flexibilita – možnost upravovat prostředí podle potřeb aplikace.
- Reálné prostředí – VPS simuluje produkční nasazení podobné praxi ve firmách.
- Nízké náklady – Oracle Cloud nabízí bezplatný provoz.

Webový server Nginx byl zvolen kvůli své vysoké výkonnosti a nízké spotřebě prostředků.

Pro běh aplikace byl použit PHP-FPM, který zajišťuje efektivní zpracování PHP požadavků.

Databáze je hostována externě pomocí služby Neon PostgreSQL, což umožňuje snadné škálování a oddělení databázové vrstvy od aplikačního serveru.

7. Architektura systému

Přehled architektury

Aplikace je navržena jako webový systém s odděleným frontendem a backendem. Architektura je založena na principu klient-server.

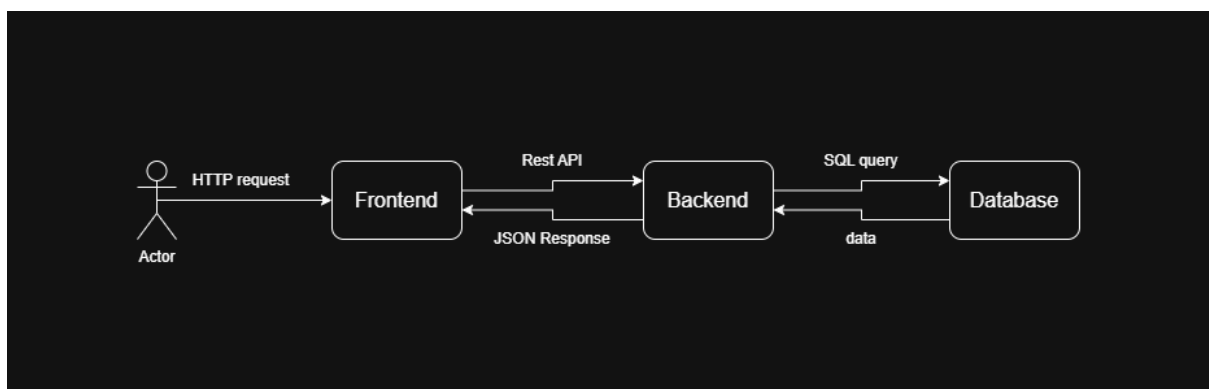
Systém se skládá ze tří hlavních komponent:

- **Frontend (klientská část)**
- **Backend API (serverová část)**
- **Databáze**

Tyto komponenty spolu komunikují pomocí HTTP požadavků a odpovědí.

Architektura je navržena jako modulární, aby umožňovala snadné rozšiřování a údržbu systému.

Diagram



Popis komponent

Frontend

Frontend představuje uživatelské rozhraní aplikace, se kterým uživatel přímo interaguje.

Odpovědnosti:

- zobrazení kvízu a otázek
- zpracování uživatelských vstupů

- odesílání požadavků na backend
- zobrazení výsledků a žebříčků

Frontend komunikuje s backendem pomocí HTTP požadavků (API).

Backend API

Backend je implementován pomocí frameworku **Symfony** a zajišťuje hlavní logiku aplikace.

Odpovědnosti:

- zpracování požadavků z frontendu
- autentizace uživatelů
- výpočet bodů
- správa otázek
- ukládání a načítání dat

Backend vystavuje API, které frontend využívá pro komunikaci.

Databáze

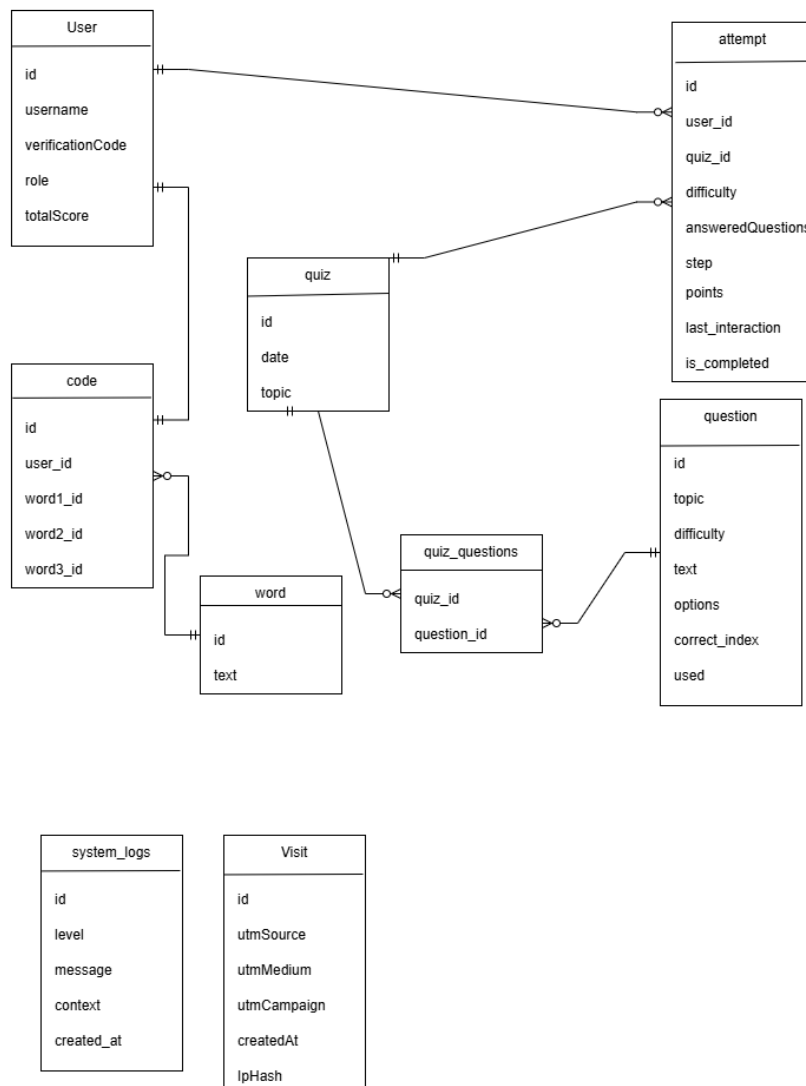
Databáze slouží pro ukládání všech důležitých dat aplikace.

Odpovědnosti:

- ukládání uživatelských účtů
- ukládání otázek
- ukládání výsledků kvízů
- ukládání bodů pro leaderboard

Backend komunikuje s databází pomocí databázových dotazů.

Datový model



8. Trasovatelnost požadavků

REQ-FUNC-1

- Use Case: UC1 Registrace uživatele
- Komponenta: Backend
- Implementace: AuthController
- Test: test_registration

REQ-FUNC-2

- Use Case: UC1 Přihlášení uživatele
- Komponenta: Backend
- Implementace: SessionAuthenticator
- Test: test_login

REQ-FUNC-3

- Use Case: UC4 Vyplnění kvízu
- Komponenta: Backend
- Implementace: QuizController
- Test: test_daily_limit

REQ-FUNC-4

- Use Case: UC4 Vyplnění kvízu
- Komponenta: Backend + frontend
- Implementace: QuizController
- Test: test_select_difficulty

REQ-FUNC-5

- Use Case: UC4 Vyplnění kvízu
- Komponenta: Backend + frontend
- Implementace: QuizController
- Test: test_get_questions

REQ-FUNC-6

- Use Case: UC9 Evidence výsledků
- Komponenta: Backend
- Implementace: QuizController
- Test: test_submit_answers

REQ-FUNC-7

- Use Case: UC5 Výpočet bodů
- Komponenta: Backend
- Implementace: QuizController
- Test: test_time_measurement

REQ-FUNC-8

- Use Case: UC5 Výpočet bodů
- Komponenta: Backend
- Implementace: QuizController
- Test: test_score_calculation

REQ-FUNC-9

- Use Case: UC9 Evidence výsledků
- Komponenta: Backend + databáze
- Implementace: Entity
- Test: test_save_result

REQ-FUNC-10

- Use Case: UC3 Změna jména uživatele
- Komponenta: Backend
- Implementace: UserController
- Test: test_update_name

REQ-FUNC-11

- Use Case: UC6 Zobrazení žebříčku
- Komponenta: Backend + frontend
- Implementace: LeaderboardController
- Test: test_leaderboard

9. Implementace

Aplikace je rozdělena na frontend a backend.

Frontend – HTML, CSS, JavaScript

- css/ – stylování (responzivní design, vlastní soubory)
- js/ – logika na straně klienta (volání API, časovač, zobrazení otázek, manipulace s DOM)

Backend – Symfony (PHP), PostgreSQL

Hlavní moduly a jejich odpovědnosti:

- **Controller/** – zpracování HTTP požadavků, routování, vrací JSON odpovědi
- **Entity/** – databázové entity
- **Repository/** – dotazy do databáze, vyhledávání a filtrování entit
- **Service/** – byznys logika (výpočet bodů, kontrola denního limitu, generování hesel)
- **ApiResource/** – definice API endpointů a jejich konfigurace
- **Command/** – CLI příkazy pro údržbu (např. naplnění databáze otázkami)
- **EventSubscriber/** – reakce na události (např. logování, odesílání emailů)
- **Security/** – autentizace
- **Logging/** – záznam událostí a chyb do logů

Databáze – PostgreSQL

Frontend komunikuje s backendem přes REST API (JSON), Symfony zpracovává požadavky, zajišťuje autentizaci, validaci a ukládání dat.

Nasazení aplikace

Aplikace je nasazována pomocí jednoduchého deploy scriptu na VPS server.

Struktura nasazení:

- releases/ – jednotlivé verze aplikace
- current/ – symbolický odkaz na aktuální verzi
- shared/ – sdílené soubory (např. .env.local)

Deploy probíhá:

1. stažení nové verze z Git repozitáře
2. instalace závislostí pomocí Composer
3. napojení konfiguračních souborů
4. vyčištění cache
5. přepnutí aktuální verze aplikace

Tento způsob umožňuje snadné vrácení na předchozí verzi v případě chyby.

10. Infrastruktura a nasazení

Aplikace je nasazena na VPS serveru v cloudové infrastruktuře Oracle Cloud.

Konfigurace serveru:

- Operační systém: Ubuntu 22.04 LTS
- Webový server: Nginx
- PHP: verze 8.3 + PHP-FPM
- Databáze: Neon PostgreSQL (externí cloudová databáze)

Nginx slouží jako webový server a reverse proxy, který zpracovává HTTP požadavky a předává je PHP-FPM.

Symfony aplikace běží v produkčním režimu (APP_ENV=prod).

Struktura nasazení:

Aplikace využívá release-based deployment:

- /var/www/myapp/releases/ – jednotlivé verze aplikace
- /var/www/myapp/current – aktivní verze (symbolický odkaz)
- /var/www/myapp/shared – sdílené soubory (např. konfigurace)

Proces nasazení:

1. vytvoření nové release složky
2. stažení projektu z Git repozitáře
3. instalace závislostí pomocí Composer
4. propojení konfiguračního souboru (.env.local)
5. vyčištění a zahřátí cache
6. nastavení oprávnění
7. přepnutí symbolického odkazu current na novou verzi
8. reload Nginx

Výhodou tohoto přístupu je možnost rychlého rollbacku na předchozí verzi.

Aplikace je dostupná přes veřejnou IP adresu a doménu:

<https://dennikvizovavyzva.online>

Bezpečnost:

- přístup na server je omezen pomocí SSH
- otevřené porty pouze 22, 80 a 443
- databáze je oddělena a běží mimo VPS

11. Evidence práce

Člen týmu	Aktivita	Požadavek	Čas
Dan Oujeský	Generování slov	REQ-FUNC-1	2
Dan Oujeský	Autentizace	REQ-FUNC-2	2
Dan Oujeský	Denní limit kvízu	REQ-FUNC-3	1
Dan Oujeský	Otázky podle obtížnosti	REQ-FUNC-4	2
Dan Oujeský	API request pro otázky	REQ-FUNC-5	4
Dan Oujeský	Zaznamenání odpovědi	REQ-FUNC-6	2
Dan Oujeský	Výpočet bodů podle času	REQ-FUNC-8	1
Dan Oujeský	Uložení výsledku	REQ-FUNC-9	2
Dan Oujeský	Změna jména uživatele	REQ-FUNC-10	1
Jakub Šrámek	Registrace uživatele	REQ-FUNC-1	2
Jakub Šrámek	Přihlášení uživatele	REQ-FUNC-2	2
Jakub Šrámek	Denní limit kvízu	REQ-FUNC-3	2
Jakub Šrámek	Volba obtížnosti	REQ-FUNC-4	1
Jakub Šrámek	Zobrazení otázek	REQ-FUNC-5	2
Jakub Šrámek	Záznam odpovědi	REQ-FUNC-6	2
Jakub Šrámek	Měření času	REQ-FUNC-7	1
Jakub Šrámek	Změna jména	REQ-FUNC-10	1
Jakub Šrámek	Celkový žebříček	REQ-FUNC-11	2

12. Ekonomické zhodnocení

1. Náklady na práci

Role	Hodinová sazba	Počet hodin	Cena
Backend Developer	700 Kč	17	11 900 Kč
Frontend Developer	600 Kč	15	9 000 Kč
Scrum master	200 Kč	9	1 800 Kč
Dokumentarista	200 Kč	10	2 000 Kč
Sysadmin	250 Kč	12	3 000 Kč

2. Náklady na infrastrukturu

Doména: 50 Kč

3. **Celkem:** 11 900 + 9000 + 1800 + 2000 + 3000 + 50 = 27 750 Kč

13. Testování (analýza stavu)

TC-FUNC-1: test_registration

1. Uživatel vstoupí na registrační stránku
2. Systém vygeneruje 3 slova
3. Ověří se vytvoření účtu a možnost přihlášení

TC-FUNC-2: test_login

1. Uživatel zadá existující přihlašovací údaje
2. Systém ověří přístup do aplikace
3. Ověří se chybové hlášení při neplatných údajích

TC-FUNC-3: test_daily_limit

1. Uživatel odehraje jeden kvíz
2. Pokusí se spustit druhý kvíz ve stejný den
3. Ověří se zobrazení informace o limitu

TC-FUNC-4: test_select_difficulty

1. Uživatel před kvízem vybere lehkou, střední nebo těžkou obtížnost
2. Ověří se, že kvíz odpovídá zvolené úrovni

TC-FUNC-5: test_get_questions

1. Uživatel spustí kvíz
2. Ověří se zobrazení 3 otázek postupně nebo najednou dle specifikace

TC-FUNC-6: test_submit_answers

1. Uživatel odpoví na otázku
2. Ověří se, že systém odpověď uložil a lze ji dohledat

TC-FUNC-7: test_time_measurement

1. Uživatel odpovídá na otázku
2. Ověří se, že systém zaznamenal čas odpovědi

TC-FUNC-8: test_score_calculation

1. Uživatel odpoví správně na otázky různé obtížnosti a času
2. Ověří se, že body odpovídají nastavenému vzorci

TC-FUNC-9: test_save_result

1. Uživatel dokončí kvíz
2. Ověří se, že výsledek je uložen v databázi a přiřazen k uživateli

TC-FUNC-10: test_update_name

1. Přihlášený uživatel zadá nové jméno
2. Ověří se aktualizace v profilu a v žebříčku

TC-FUNC-11: test_leaderboard

1. Uživatel přejde na stránku žebříčku
2. Ověří se zobrazení seřazených uživatelů podle bodů

TC-QUAL-1: test_availability

- Aplikace je dostupná v různých časech během celého dne

TC-QUAL-2: test_ui_clarity

- Uživatel (ideálně nový) dostane za úkol najít žebříček a spustit kvíz bez návodu
- Sleduje se intuitivnost ovládání a čitelnost textů
- Ověří se, že UI neobsahuje matoucí prvky a je vizuálně konzistentní

TC-QUAL-3: test_response_time

- Uživatel provede akci (např. odeslání odpovědi nebo načtení profilu)
- Měří se čas od kliknutí po kompletní vykreslení odpovědi/stránky
- Ověří se, že časová prodleva není příliš velká

TC-QUAL-4: test_browser_compatibility

- Uživatel otevře aplikaci postupně v Chrome, Firefox, Safari a Edge
- Zkontroluje se funkčnost kvízu a zobrazení žebříčku v každém z nich
- Ověří se, že vzhled (layout) zůstává ve všech prohlížečích nerozbitý

TC-QUAL-5: test_content_appropriateness

- Kontrola sady otázek a generovaných slov na přítomnost vulgarismů nebo nevhodných témat
- Prověření uživatelských jmen v žebříčku (případná existence filtrů na nadávky)
- Ověří se, že veškerý viditelný obsah je v souladu se školním prostředím

Aktuální stav kvality aplikace

Aplikace prošla úspěšným manuálním testováním dle 11 testovacích scénářů, které pokrývají veškerou funkčnost od registrace až po zobrazení žebříčku. Všechny scénáře byly vyhodnoceny jako vyhovující.

Výsledky auditu nástrojem Lighthouse dosahují hodnot: **Performance 100**, **Accessibility 91**, **Best Practices 100** a **SEO 90**, což potvrzuje vysokou kvalitu z hlediska rychlosti, přístupnosti i optimalizace pro vyhledávače. V současné době nejsou evidovány žádné kritické chyby, aplikace je připravena k nasazení.

14. Naměřená data z logů

Analýza byla provedena nad logy webového serveru Nginx (/var/log/nginx/myapp_access.log).

Zjištěné údaje:

- Celkový počet požadavků: 24 982
- Počet unikátních IP adres: 1705
- Počet HTTP 4xx chyb: 19 395
- Počet HTTP 5xx chyb: 55

Časová analýza:

Nejvyšší provoz byl zaznamenán v ranních hodinách (okolo 6:00), kde bylo zaznamenáno nejvíce požadavků.

Naopak nejnižší provoz byl v časných ranních hodinách (okolo 5:00).

Tento rozdíl může být způsoben automatickými požadavky (např. boty nebo crawleři) nebo časem, kdy uživatelé začínají používat aplikaci.

Analýza ukázala, že většina chyb 404 vzniká kvůli neexistujícím URL nebo chybným požadavkům.

Chyby 500 nebyly významně zastoupeny, což naznačuje stabilní backend.

Logy byly analyzovány pomocí nástroje GoAccess.

15. Zhodnocení projektu

Co se podařilo

V rámci projektu se podařilo úspěšně implementovat hlavní funkcionalitu aplikace dle původního návrhu.

Aplikace umožňuje uživateli:

- registrovat se do systému pomocí vygenerovaných přihlašovacích údajů,
- přihlásit se ke svému účtu,
- upravit své uživatelské jméno,
- odehrát kvíz skládající se ze tří otázek.

Byl implementován bodovací systém, který zohledňuje:

- obtížnost zvoleného kvízu,
- rychlost odpovědí uživatele.

Na základě získaných bodů je uživatel zařazen do **globálního žebříčku**, který je plně funkční a umožňuje porovnání výsledků mezi jednotlivými hráči.

Z technického hlediska se podařilo:

- propojit backend aplikace s databází,
- zajistit ukládání uživatelských dat a výsledků kvízů,
- implementovat komunikaci mezi backendem a frontendem

Celkově se podařilo vytvořit funkční prototyp aplikace, který splňuje hlavní cíle projektu a umožňuje jeho další rozšiřování.

Co bylo problematické

Během vývoje se objevil problém s výkonem aplikace, konkrétně při komunikaci s databází.

Původní řešení využívalo přístup k externí databázi způsobem, který nebyl dostatečně optimalizovaný. Backend při každém požadavku navazoval komunikaci s databází bez efektivní správy připojení, což vedlo k vyšší latenci a pomalé odezvě aplikace.

Tento problém se projevoval zejména při načítání otázek a ukládání výsledků, kdy reakce systému trvala déle, než bylo žádoucí.

Řešením bylo přepracování komunikace mezi backendem a databází. Byla zavedena efektivnější práce s databázovým připojením a optimalizace dotazů. Backend nyní využívá standardní přístup přes databázovou vrstvu (repository/service), což výrazně zlepšilo výkon a stabilitu aplikace.

Možnosti dalšího rozvoje

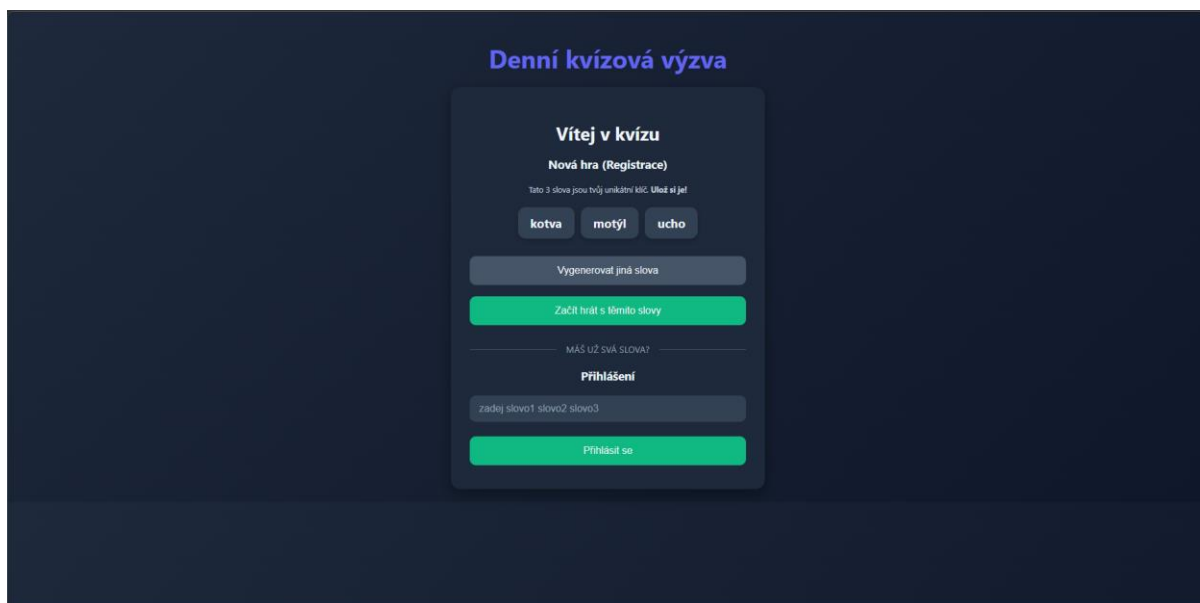
Aplikaci by bylo možné v budoucnu dále rozšiřovat o nové funkce a vylepšení. Mezi hlavní možnosti patří:

- **Rozšíření typů otázek**
Přidání různých typů otázek (např. výběr z více možností, pravda/nepravda, obrázkové otázky), což by zvýšilo variabilitu kvízů.
- **Denní žebříček (Daily leaderboard)**
Zavedení denního žebříčku, který by se každý den resetoval a zvýšil motivaci k pravidelnému hraní.
- **Více otázek a kategorií**
Rozdělení otázek do tematických kategorií (např. IT, škola, obecné znalosti) a možnost výběru tématu.
- **Vylepšení uživatelského rozhraní**
Modernizace designu aplikace, lepší přehlednost a responzivita pro mobilní zařízení.
- **Multiplayer nebo soutěže**
Možnost soutěžit přímo proti jiným uživatelům v reálném čase.
- **Bezpečnostní vylepšení**
Například lepší ochrana účtů nebo validace vstupů.

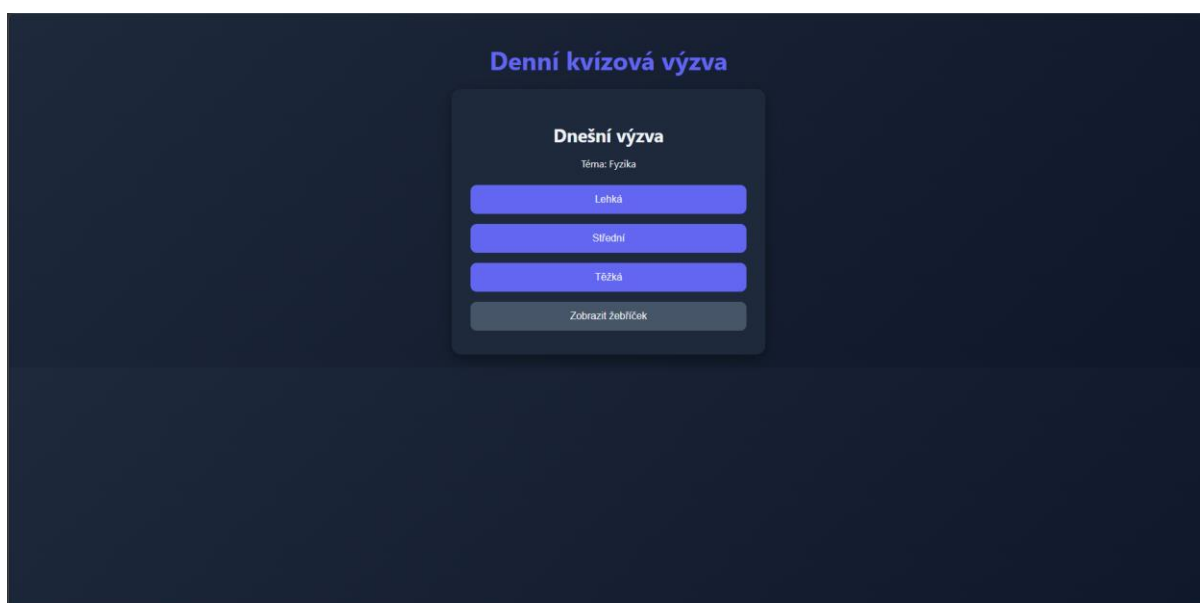
16. Přílohy

Odkaz na aplikaci: <https://dennikvizovavyzva.online/>

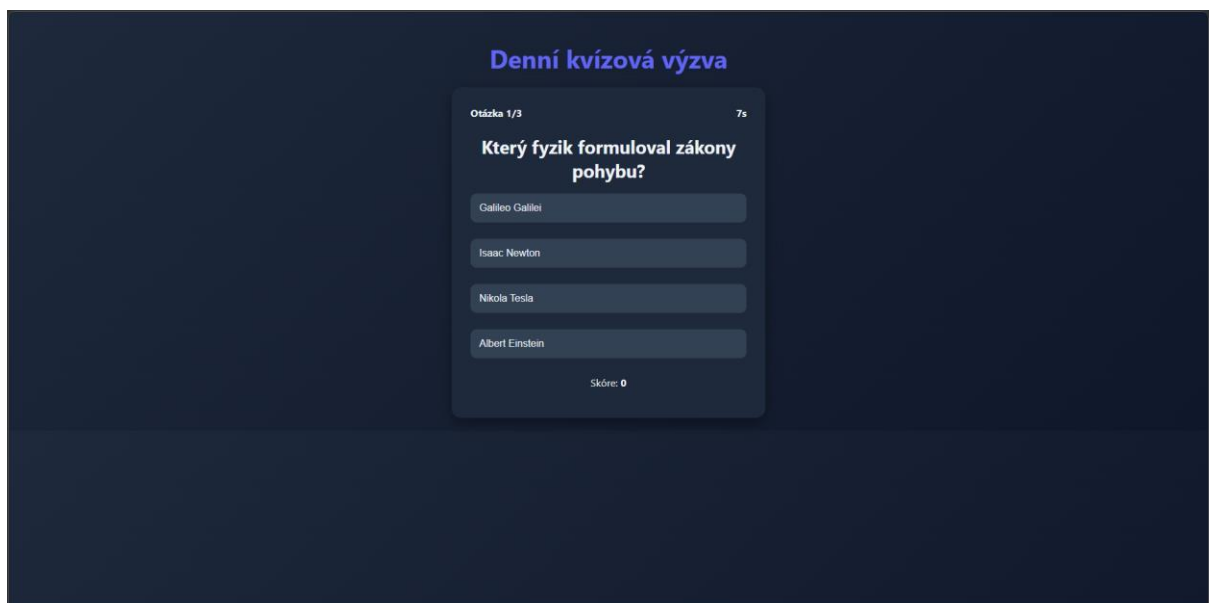
Odkaz na repozitář: [NQKhanh296/Daily-Quiz-Challenge: Každodenní krátké kvízy pro studenty](#)



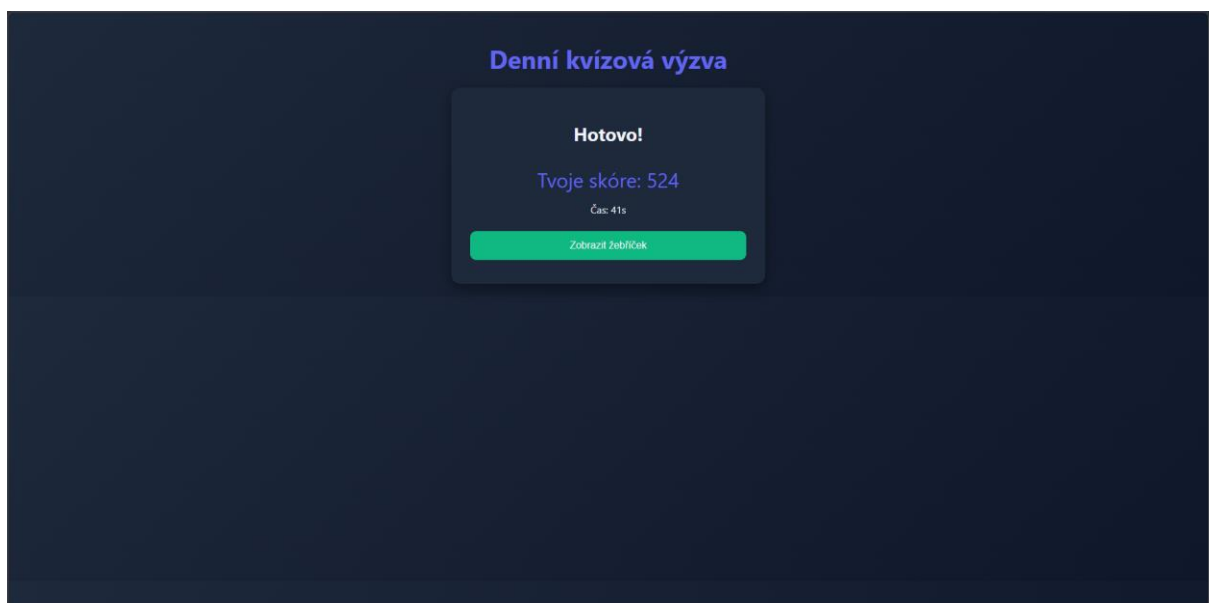
Obr. 1: Přihlašovací stránka aplikace



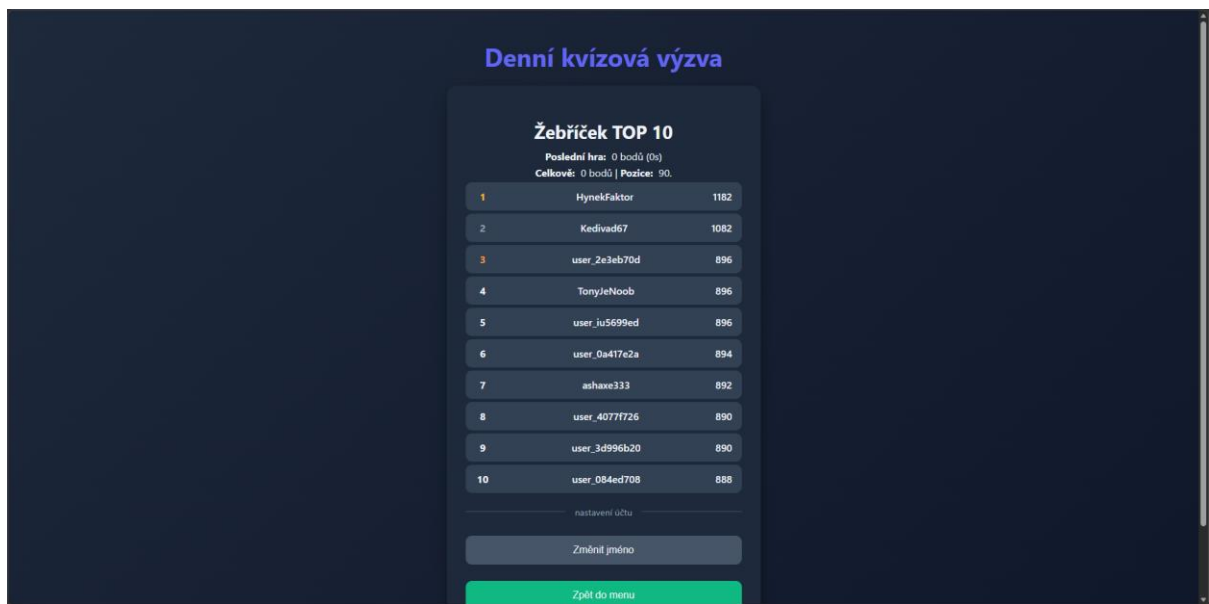
Obr. 2: Stránka s výběrem obtížnosti kvízu



Obr. 3: Otázky kvízu



Obr. 4: Vyhodnocení bodů po vyplnění kvízu



Obr. 5: Žebříček všech hráčů